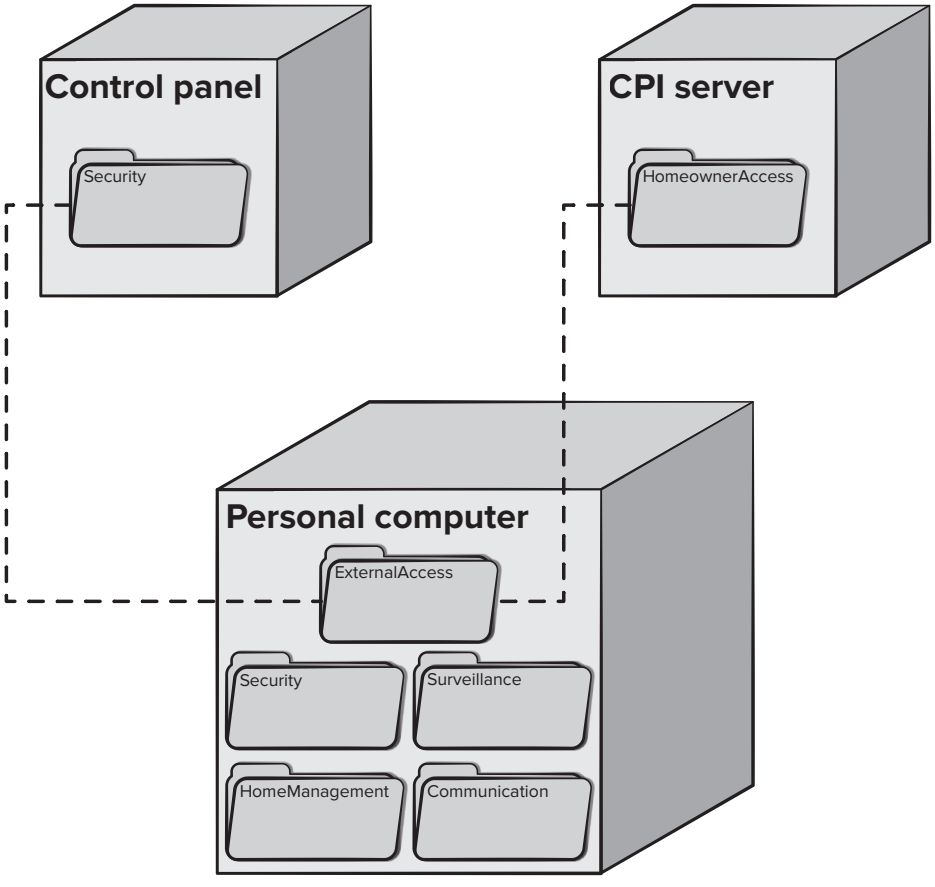
**FIGURE 9.7**
**A UML deployment diagram**




The diagram shown in Figure 9.7 is in *descriptor form.* This means that the deployment diagram shows the computing environment but does not explicitly indicate configuration details. For example, the "personal computer" is not further identified. It could be a Mac, a Windows-based PC, a Linux box, or a mobile platform with its associated operating system. These details are provided when the deployment diagram is revisited in *instance form* during the latter stages of design or as construction begins. Each instance of the deployment (a specific, named hardware configuration) is identified.

## 9.5 SUMMARY

Software design commences as the first iteration of requirements engineering concludes. The intent of software design is to apply a set of principles, concepts, and practices that lead to the development of a high-quality system or product. The goal of design is to create a model of software that will implement all customer requirements

correctly and bring delight to those who use it. Software designers must sift through many design alternatives and converge on a solution that best suits the needs of project stakeholders.

The design process moves from a "big picture" view of software to a narrower view that defines the detail required to implement a system. The process begins by focusing on architecture. Subsystems are defined, communication mechanisms among subsystems are established, components are identified, and a detailed description of each component is developed. In addition, external, internal, and user interfaces are designed at the same time.

Design concepts have evolved over the first 60 years of software engineering work. They describe attributes of computer software that should be present regardless of the software engineering process that is chosen, the design methods that are applied, or the programming languages that are used. In essence, design concepts emphasize the need for abstraction as a mechanism for creating reusable software components; the importance of architecture as a way to better understand the overall structure of a system; the benefits of pattern-based engineering as a technique for designing software with proven capabilities; the value of separation of concerns and effective modularity as a way to make software more understandable, more testable, and more maintainable; the consequences of information hiding as a mechanism for reducing the propagation of side effects when errors do occur; the impact of functional independence as a criterion for building effective modules; the use of refinement as a design mechanism; the application of refactoring for optimizing the design that is derived; the importance of object-oriented classes and the characteristics that are related to them; the need to use abstraction to reduce coupling between components; and the importance of design for testing.

The design model encompasses four different elements. As each of these elements is developed, a more complete view of the design evolves. The architectural element uses information derived from the application domain, the requirements model, and available catalogs for patterns and styles to derive a complete structural representation of the software, its subsystems, and components. Interface design elements model external and internal interfaces and the user interface. Component-level elements define each of the modules (components) that populate the architecture. Finally, deployment-level design elements allocate the architecture, its components, and the interfaces to the physical configuration that will house the software.

## PROBLEMS AND POINTS TO PONDER

**9.1.** Do you design software when you "write" a program? What makes software design different from coding?

**9.2.** If a software design is not a program (and it isn't), then what is it?

**9.3.** How do we assess the quality of a software design?

**9.4.** Describe software architecture in your own words.

**9.5.** Describe separation of concerns in your own words. Is there a case when a "divide and conquer" strategy may not be appropriate? How might such a case affect the argument for modularity?

**9.6.** Discuss the relationship between the concept of information hiding as an attribute of effective modularity and the concept of module independence.

**9.7.** How are the concepts of coupling and software portability related? Provide examples to support your discussion.

**9.8.** Apply a "stepwise refinement approach" to develop three different levels of procedural abstractions for one or more of the following programs: (1) Develop a check writer that, given a numeric dollar amount, will print the amount in words normally required on a check. (2) Iteratively solve for the roots of a transcendental equation. (3) Develop a simple task-scheduling algorithm for an operating system.

**9.9.** Does *refactoring* mean that you modify the entire design iteratively? If not, what does it mean?

**9.10.** Briefly describe each of the four elements of the design model.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

# 10

# ARCHITECTURAL DESIGN—A RECOMMENDED APPROACH

Design has been described as a multistep process in which representations of data and program structure, interface characteristics, and procedural detail are synthesized from information requirements. As we noted in Chapter 9, design is information driven. Software design methods are derived from consideration of each of the three domains of the analysis model. The decisions made while considering the data, functional, and behavioral domains serve as guides for the creation of the software architectural design.

**KEY CONCEPTS**


agility and architecture . . . . . 185
archetypes . . . . . 196
architectural considerations . . . . . 193
architectural decisions . . . . . 195
architectural description language . . . . . 184
architectural descriptions . . . . . 184
architectural design . . . . . 196
architectural patterns . . . . . 187
architectural styles . . . . . 186
architecture . . . . . 182
architecture conformance checking . . . . . 204
architecture trade-off analysis method (ATAM) . . . . . 201
layered architectures . . . . . 189
refining the architecture . . . . . 198
taxonomy of architectural styles . . . . . 187




## QUICK LOOK

**What is it?** Architectural design represents the structure of data and program components that are required to build a computer-based system. It considers the architectural style that the system will take, the structure and properties of the components that constitute the system, and the interrelationships that occur among all architectural components of a system.

**Who does it?** Although a software engineer can design both data and architecture, the job is often allocated to specialists when large, complex systems are to be built. A database or data warehouse designer creates the data architecture for a system. The "system architect" selects an appropriate architectural style from the requirements derived during software requirements analysis.

**Why is it important?** You wouldn't attempt to build a house without a blueprint, would you? You also wouldn't begin drawing blueprints by sketching the plumbing layout for the house. You'd need to look at the big picture—the house itself—before you worry about details. That's what architectural design does—it provides you with the big picture and ensures that you've got it right.

**What are the steps?** Architectural design begins with data design and then proceeds to the derivation of one or more representations of the architectural structure of the system. Alternative architectural styles or patterns are analyzed to derive the structure that is best suited to customer requirements and quality attributes. Once an alternative has been selected, the architecture is elaborated, using an architectural design method.

**What is the work product?** An architecture model encompassing data architecture and program structure is created during architectural design. In addition, component properties and relationships (interactions) are described.

**How do I ensure that I've done it right?** At each stage, software design work products are reviewed for clarity, correctness, completeness, and consistency with requirements and with one another.

Philippe Kruchten, Grady Booch, Kurt Bittner, and Rich Reitman [Mic09] suggest that software architecture identifies a system's "structural elements and their interfaces," along with the "behavior" of individual components and subsystems. They write that the job of architectural design is to create "coherent, well-planned representations" of the system and software.

Methods to create such representations of the data and architectural layers of the design model are presented in this chapter. The objective is to provide a systematic approach for the derivation of the architectural design—the preliminary blueprint from which software is constructed.

## 10.1 SOFTWARE ARCHITECTURE

In their landmark book on the subject, Shaw and Garlan [Sha15] argue that since the earliest days of computer programming, "software systems have had architectures, and programmers have been responsible for the interactions among the modules and the global properties of the assemblage." Today, effective software architecture and its explicit representation and design have become dominant themes in software engineering.

### 10.1.1 What Is Architecture?

When you consider the architecture of a building, many different attributes come to mind. At the most simplistic level, you think about the overall shape of the physical structure. But in reality, architecture is much more. It is the manner in which the various components of the building are integrated to form a cohesive whole. It is the way in which the building fits into its environment and meshes with other buildings in its vicinity. It is the degree to which the building meets its stated purpose and satisfies the needs of its owner. It is the aesthetic feel of the structure—the visual impact of the building—and the way textures, colors, and materials are combined to create the external facade and the internal "living environment." It is small details—the design of lighting fixtures, the type of flooring, the placement of wall hangings; the list is almost endless. And finally, it is art.

Architecture is also something else. It is "thousands of decisions, both big and small" [Tyr05]. Some of these decisions are made early in design and can have a profound impact on all other design actions. Others are delayed until later, thereby eliminating overly restrictive constraints that would lead to a poor implementation of the architectural style.

Just like the plans for a house are merely a representation of the building, the software architecture representation is not an operational product. Rather, it is a representation that enables you to (1) analyze the effectiveness of the design in meeting its stated requirements, (2) consider architectural alternatives at a stage when making design changes is still relatively easy, and (3) reduce the risks associated with the construction of the software.

This definition emphasizes the role of "software components" in any architectural representation. In the context of architectural design, a software component can be something as simple as a program module or an object-oriented class, but it can also be extended to include databases and "middleware" that enable the configuration of

a network of clients and servers. The properties of components are those characteristics that are necessary to an understanding of how the components interact with other components. At the architectural level, internal properties (e.g., details of an algorithm) are not specified. The relationships between components can be as simple as a procedure call from one module to another or as complex as a database access protocol.

We believe that a software design can be thought of as an instance of some software architecture. However, the elements and structures that are defined as parts of particular software architectures are the root of every design. It is our recommendation that design should begin with a consideration of the software architecture.

### 10.1.2 Why Is Architecture Important?

In a book dedicated to software architecture, Bass and his colleagues [Bas12] identify three key reasons that software architecture is important:

- Software architecture provides a representation that facilitates communication among all stakeholders.
- The architecture highlights early design decisions that will have a profound impact on all software engineering work that follows.
- The architecture constitutes a relatively small model of how the system components are structured and work together.

The architectural design model and the architectural patterns contained within it are transferable. That is, architecture genres, styles, and patterns (Sections 10.3 through 10.6) can be applied to the design of other systems and represent a set of abstractions that enable software engineers to describe architecture in predictable ways.

Making good decisions while defining the software architecture is critical to the success of a software product. The software architecture sets the structure of the system and determines its quality [Das15].

### 10.1.3 Architectural Descriptions

Each of us has a mental image of what the word *architecture* means. The implication is that different stakeholders will see a given software architecture from different viewpoints that are driven by different sets of concerns. This implies that an architectural description is actually a set of work products that reflect different views of the system.

Smolander, Rossi, and Purao [Smo08] have identified multiple metaphors, representing different views of the same architecture that stakeholders use to understand the term *software architecture*. The *blueprint metaphor* seems to be most familiar to the stakeholders who write programs to implement a system. Developers regard architecture descriptions as a means of transferring explicit information from architects to designers to software engineers charged with producing the system components.

The *language metaphor* views architecture as a facilitator of communication across stakeholder groups. This view is preferred by stakeholders with a high customer focus (e.g., managers or marketing experts). The architectural description needs to be concise and easy to understand because it forms the basis for negotiation, particularly in determining system boundaries.